



ZAPX

EXTENDING OWASP ZAP FOR ENHANCED WEB VULNERABILITY DETECTION AND AI- POWERED REMEDIATION

PRESENTED BY :

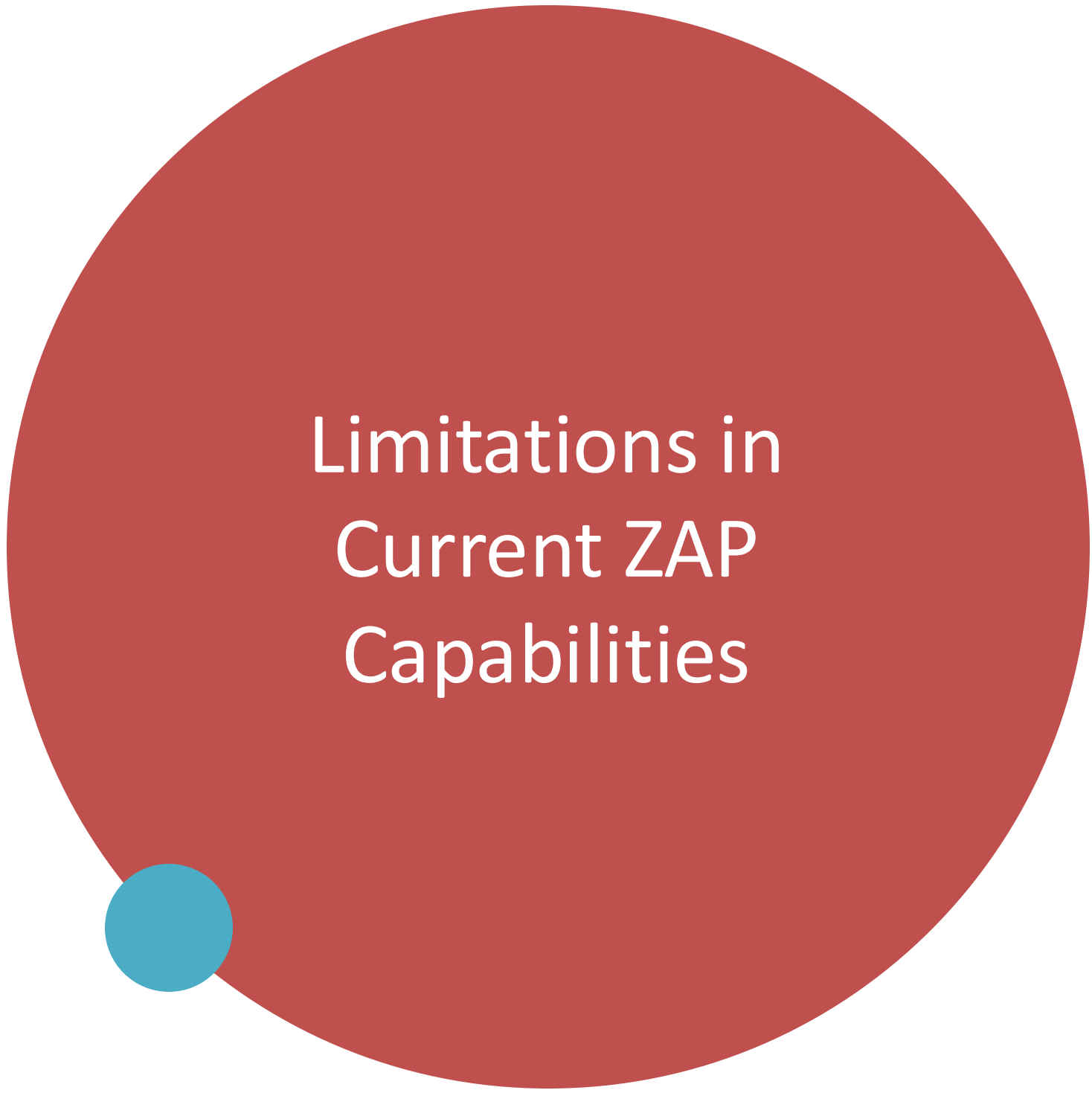
ARCHAWIT CHANGTOR
PAKAPON RATTANASRISUK
KITTIPAT TANGTANAWIRUT
SOMCHART FUGKEAW

**Sirindhorn International Institute of Technology, Thammasat University,
Thailand**



Background and Motivation

- OWASP **Zed Attack Proxy (ZAP)** — open-source penetration testing tool for web app vulnerabilities (OWASP Top 10).
- Works as a **proxy** to intercept and inspect HTTP(S) traffic between browser and target.
- Detects traditional vulnerabilities: **SQL Injection, XSS, CSRF, IDOR, Input Validation, Insecure Authentication.**
- **Challenge:** Modern attacks increasingly leverage **automation** (web scraping, brute force, request flooding).
- Existing ZAP modules lack coverage for **automated threats** and **client-side exposures.**



Limitations in Current ZAP Capabilities

1. Automated Attack Detection Gaps:

Limited visibility into automation-based vulnerabilities (e.g., rate-limiting, bot defense).

2. Client-Side Security Weakness:

Exposure of API keys, tokens, secrets in JavaScript or browser storage.

3. Remediation and Verification Challenges:

Developers left to interpret findings manually.

High false-positive rates reduce trust and slow response.

OWASP ZAP?

- Free, open-source VA
- Web application level
- Automated or Manual scan
- Vulnerability alerts with some information

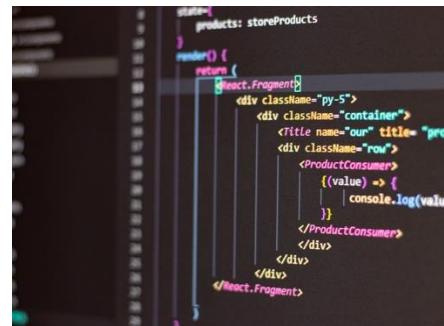


ZAPX

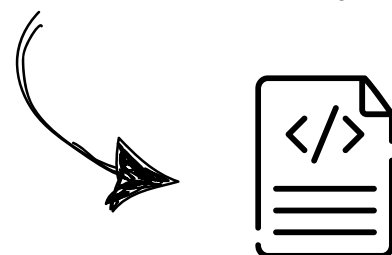
However,



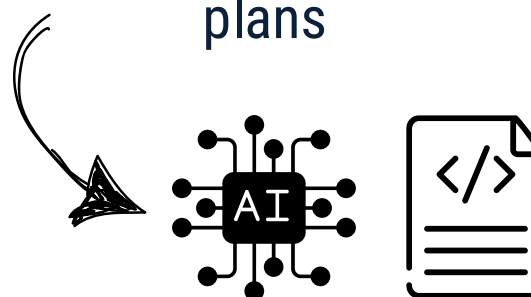
NO Automated bot defense detection



Partial Client-side code vulnerability detection



Impractical mitigation plans



SIIT
SINCE 1992

Related Work

- Vulnerability detection research has advanced across:

- Scanner enhancement
- Penetration-testing automation
- Machine-learning–driven security analysis
- Systematic evaluations of tools and usability

- Prior works aim to improve automation and efficiency in:

- Vulnerability scanning [9, 15]
- Penetration testing [4, 16]

- Limitations of Existing Studies

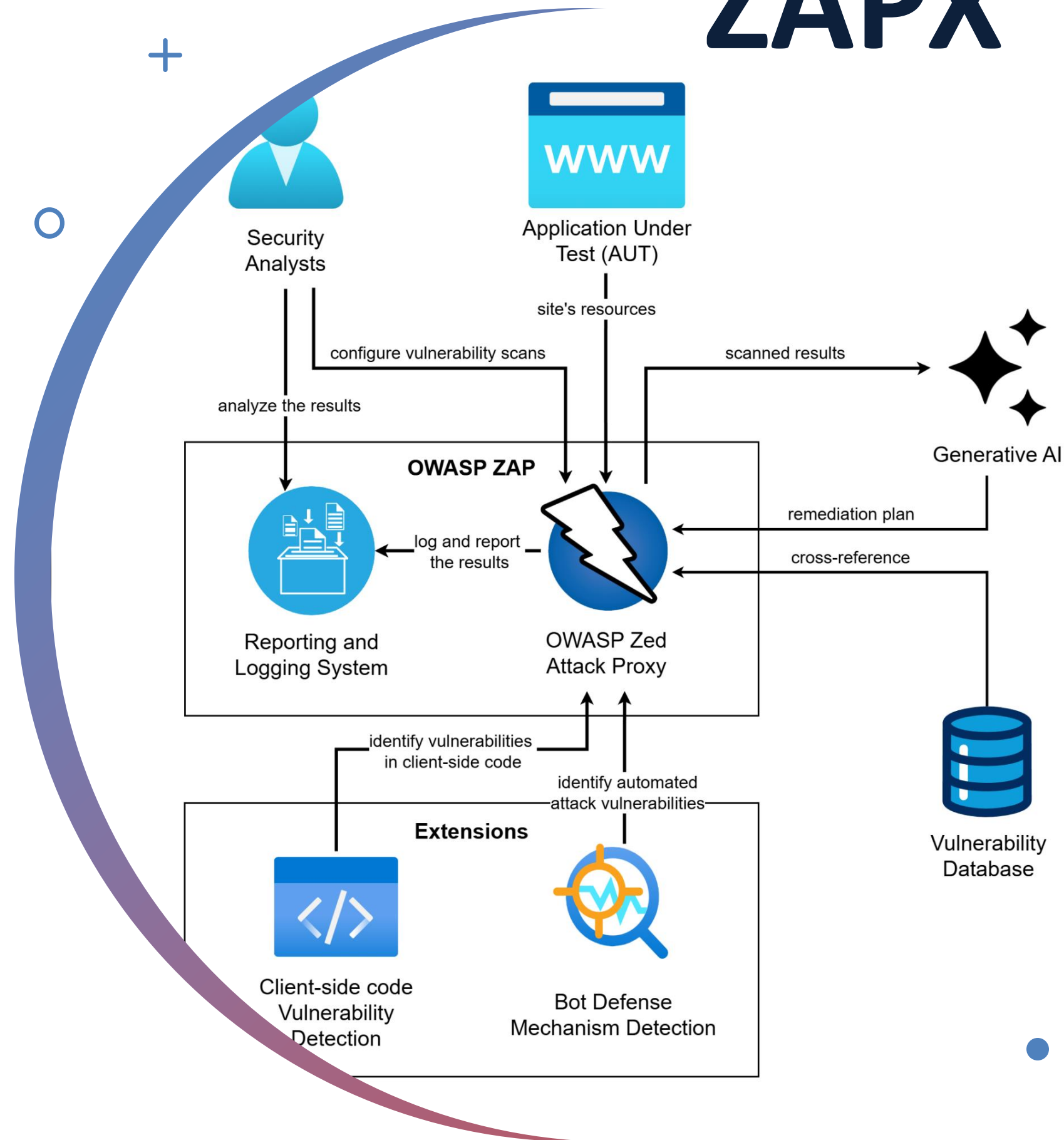
- Most works address:

- Detection **only**, or
- Efficiency optimization **only**

- Lack of **holistic solutions** that unify:

- Detection
- Automated exploitation
- Developer-centric remediation

ZAPX



- **OWASP Zed Attack Proxy (ZAP)** is the primary platform used for security testing. It intercepts, inspects, and modifies HTTP/S requests and responses between the tester's browser and the web application to detect vulnerabilities such as SQL injections and cross-site scripting.
- **Automated Attack Vulnerability Detection Extension** extends ZAP's functionality by **identifying vulnerabilities susceptible to automated attacks**. It analyzes **application behavior and uses rule-based detection to detect vulnerabilities like web scraping and brute force attacks**.
- **Client-Code Vulnerability Detection** enhances ZAP's capabilities by focusing on vulnerabilities in the client-side code of the web application. This includes **scanning HTML, JavaScript, and other resources to identify issues such as exposed sensitive information, insecure client-side storage, and vulnerable client-side logic**.
- **Generative AI** **produces context-specific solutions for the detected vulnerabilities, streamlining the remediation process**. It analyzes the specific context of the vulnerability and generates tailored, step-by-step instructions based on the application's code and technology stack.
- **Vulnerability Databases** provide the authoritative source of known vulnerabilities. They are integrated into the system, allowing AI-powered remediation plans to cross-reference and ensure up-to-date scanning results.
- **Security Analysts** use ZAP and its enhanced tools to identify, analyze, and mitigate vulnerabilities. They configure scanning parameters, interpret results, and apply remediation strategies generated by the system to address security issues.
- **Application Under Test (AUT)** is the web application being assessed. ZAP interacts with the AUT during scans, analyzing the data it provides and responses for potential vulnerabilities.
- **Reporting and Logging System** manages the collection of logs and the generation of reports throughout the scanning lifecycle. It aggregates data from all components, tracks exploit results, and produces comprehensive reports for stakeholders. An audit trail is maintained for compliance and future reference.

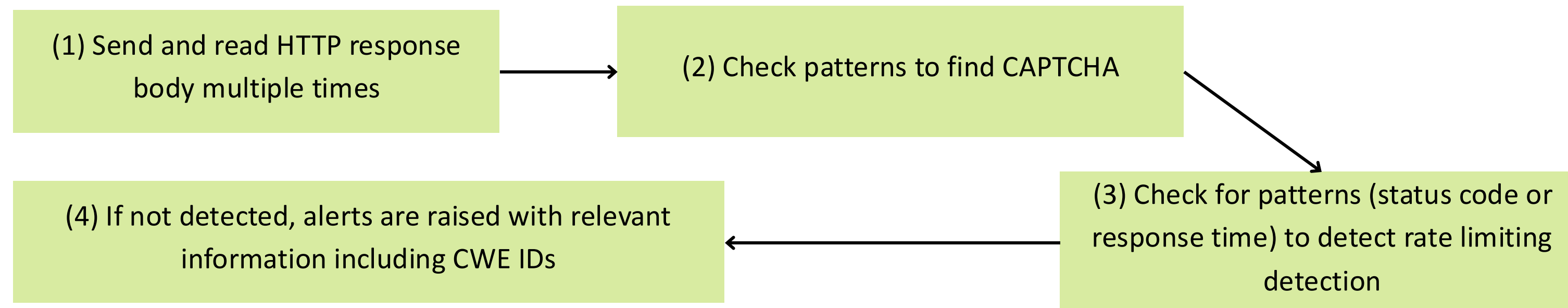
OUR APPROACH Phase 1

OBJECTIVE

- To detect **automated bot defense mechanisms vulnerability** using ZAP

SOLUTIONS

- Add an **active rule script** into ZAP (run on active scan)



Algorithm 1: Detecting Bot Defense Mechanisms

BEGIN

FUNCTION send_request(target_url):

 Send HTTP GET request

 Record response time, status code, and body

 If "captcha" found → captcha_detected = True

ENDFUNCTION

FUNCTION analyze_results():

 Compare response times → detect rate limiting

 Inspect status codes (403, 429) → detect blocking

 Search response content → detect CAPTCHA

 Generate alerts with CWE/WASC references

ENDFUNCTION

FUNCTION main():

 For i in 1..number_of_requests:

 CALL send_request(target_url)

 CALL analyze_results()

ENDFUNCTION

Initialize lists for response_times, response_statuses

Set captcha_detected = False

CALL main()

END

- The bot detection mechanism operates by **measuring response times, inspecting HTTP status codes, and scanning response bodies for CAPTCHA patterns.**
- The idea is to determine whether the application employs effective countermeasures:
- Increased response times may indicate rate limiting.
- HTTP status codes (403, 429) may suggest blocking.
- Response content containing “captcha” may reveal a challenge-response defense.
- If no defenses are observed, the script flags the application as lacking adequate protection against automated threats.

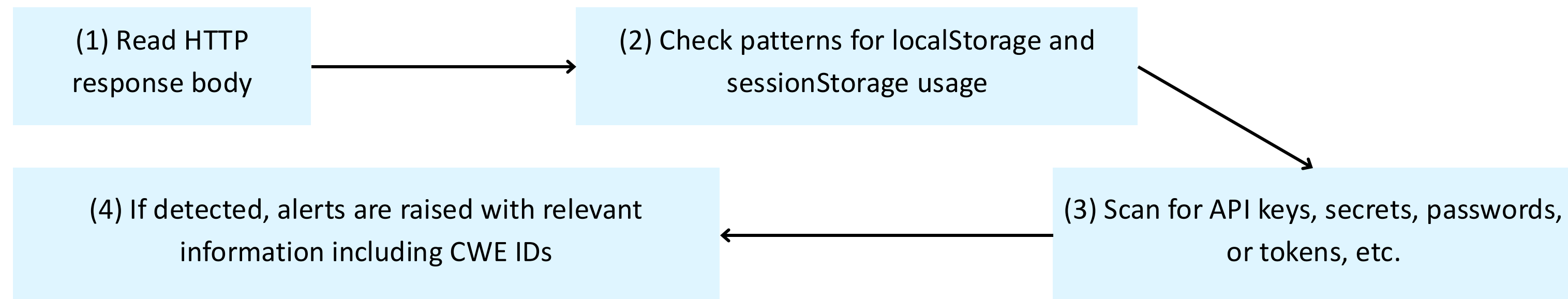
OUR APPROACH –Phase 2

OBJECTIVE

- To identify **insecure use of client-side storage vulnerability** using ZAP
- To detect **exposed sensitive information in client-side code** using ZAP

SOLUTIONS

- Add a **passive rule script** into ZAP (rule-based detection)



Algorithm 2: Identifying insecure use of client-side code

```
BEGIN
FUNCTION scan():
  If response.status = 200 AND content-type ∈ {HTML, JavaScript}:
    Extract response_body
    CALL analyze_insecure_storage(response_body)
    CALL analyze_sensitive_info(response_body)

FUNCTION analyze_insecure_storage(body):
  Search for localStorage/sessionStorage usage
  For each match → Raise alert "Insecure Client-Side Storage" (CWE-922, WASC-15)

FUNCTION analyze_sensitive_info(body):
  Search for API keys, tokens, passwords, secrets
  For each match → Raise alert "Exposed Sensitive Information" (CWE-532, WASC-13)

FUNCTION main():
  CALL scan()
ENDFUNCTION

CALL main()
END
```

- First, response filtering ensures that only valid responses with a 200 (OK) status code and content types of HTML or JavaScript are processed.
- Next, the script **performs pattern analysis by scanning the response body** to detect insecure storage operations, such as the use of localStorage or sessionStorage, and to identify exposed sensitive information, including API keys, tokens, and passwords.
- Finally, when potential vulnerabilities are found, **the system performs alert generation, issuing alerts that include detailed descriptions, supporting evidence, and standard CWE/WASC references.**

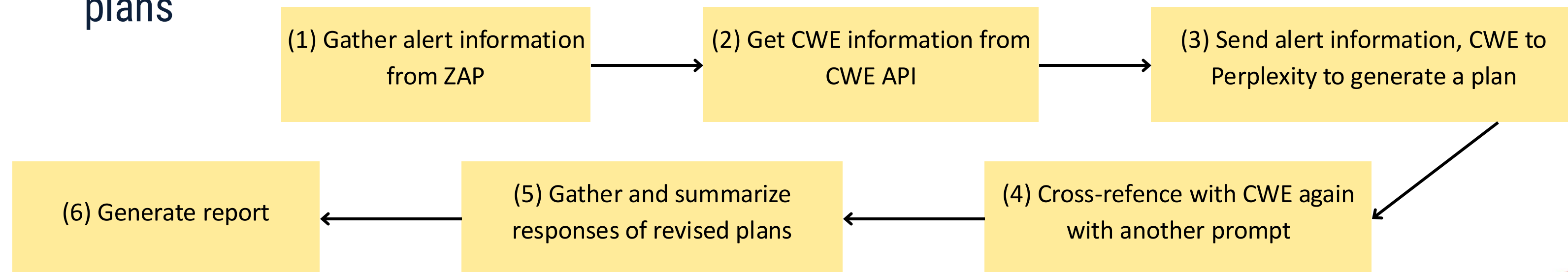
OUR APPROACH –Phase 3

OBJECTIVE

- To generate **detailed and practical remediation plans** from generative AI

SOLUTIONS

- Add a **stand alone script** into ZAP (trigger manually)
- **Use Perplexity API** to generate actionable, context-specific remediation plans
- **Cross-reference with CWEs** to validate and provide more accuracy for remediation plans



Algorithm 3: Generating a remediation plan

```
BEGIN
FUNCTION get_zap_alerts():
    Retrieve alerts from ZAP API
    Return high-risk alerts

FUNCTION get_cwe_details(cwe_id):
    Query CWE API for weakness description & mitigations
    Return CWE details

FUNCTION get_mitigation_plan(alert, cwe_details):
    Send alert + CWE info + tech stack to generative AI
    Receive remediation plan

FUNCTION recheck_mitigation_plan(plan, cwe_details):
    Validate AI response against CWE details
    Return refined remediation plan

FUNCTION process_alerts(alerts):
    For each alert:
        Retrieve CWE details
        Generate and validate remediation plan
        Append to final_plans

FUNCTION generate_report(final_plans):
    Compile markdown report with alerts + remediation plans
    Save to OUTPUT_PATH

MAIN:
    alerts = get_zap_alerts()
    final_plans = process_alerts(alerts)
    generate_report(final_plans)
END
```

- First, alerts are retrieved via the ZAP API [10], filtering for significant risks.
- Second, **CWE details corresponding to each alert are queried**, providing contextual descriptions and potential mitigations.
- Third, **the alert data and CWE details are passed to the generative AI model to produce remediation plans** tailored to the application's technology stack.
- Fourth, **the AI response is revalidated by cross-checking against CWE data to refine and confirm the mitigation steps**.
- Finally, a report is generated, consolidating the validated remediation plans into a markdown file for developers, analysts, and auditors.

PHASE 4: Automated Scanning using OWASP ZAP

To enable the automated scanning through ZAPx end-to-end and validate the custom scripts.

- First, we configure ZAP's Scan Policy to enable Script Active Scan Rules (and relevant passive rules), ensuring the scan focuses on automated-attack vectors and client-side risks.
- We then execute an Automated Scan against the Application Under Test (AUT), during which our Phase-1/Phase-2 scripts emit normalized alerts.
- Next, we invoke the Standalone Remediation module (Phase-3) to aggregate alerts, enrich them with CWE details, and generate context-specific mitigation plans via the generative AI engine.
- Finally, we compile a single markdown report containing findings, evidence, CWE/WASC references, and actionable fixes for developers and auditors.

EVALUATION AND IMPLEMENTATION

- *Functionality Comparison*

Capabilities / Detection	ZAP	ZAPx
Insecure use of client-side storage	X	✓
Exposed sensitive information in client-side code	Partial	✓
Bot defense mechanism (i.e. rate limiting, CAPTCHA, IP blocking)	X	✓
Detailed and practical mitigation plan	Partial	✓

Simulation

We evaluated ZAPx on **two simulated web applications**:

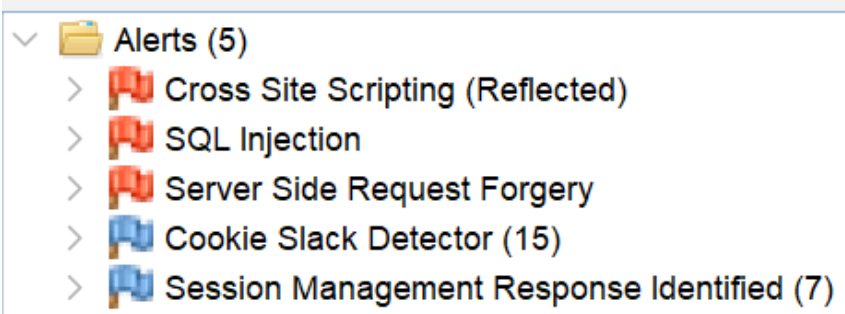
- 1. Vulnerable Web Application** – containing six intentional vulnerabilities: Cross-Site Scripting (XSS), SQL Injection (SQLi), Server-Side Request Forgery (SSRF), insecure client-side storage, exposed secrets in client-side code, and missing bot defenses.
- 2. Patched Web Application** – a secure version in which all vulnerabilities were mitigated based on remediation plans generated by AI.

Both applications were developed using **Node.js** [17], **Express** [18], and **MySQL** reflecting a common modern web stack.

Our evaluation consists of three parts: **client-side vulnerability detection, bot defense mechanism detection, and remediation plan validation**

TESTING OUR SCRIPTS

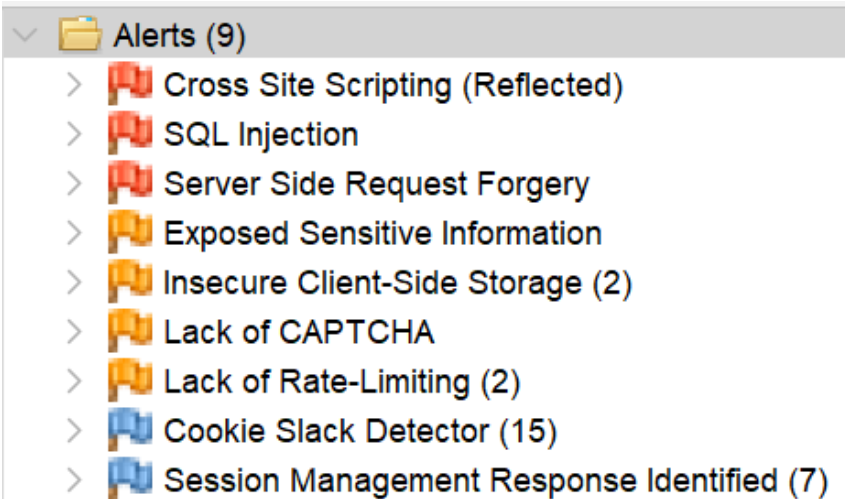
- **Run crawler and active scan** to find vulnerabilities and gather alerts and useful information for generating a remediation plan



Alerts (5)

- > Cross Site Scripting (Reflected)
- > SQL Injection
- > Server Side Request Forgery
- > Cookie Slack Detector (15)
- > Session Management Response Identified (7)

ZAP

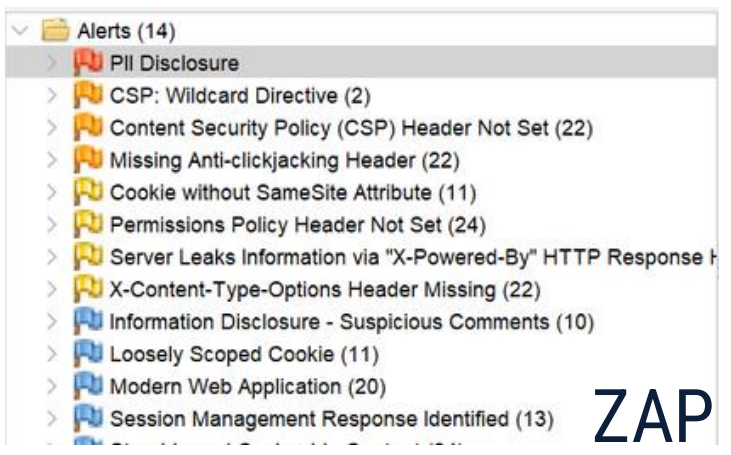


Alerts (9)

- > Cross Site Scripting (Reflected)
- > SQL Injection
- > Server Side Request Forgery
- > Exposed Sensitive Information
- > Insecure Client-Side Storage (2)
- > Lack of CAPTCHA
- > Lack of Rate-Limiting (2)
- > Cookie Slack Detector (15)
- > Session Management Response Identified (7)

ZAPx

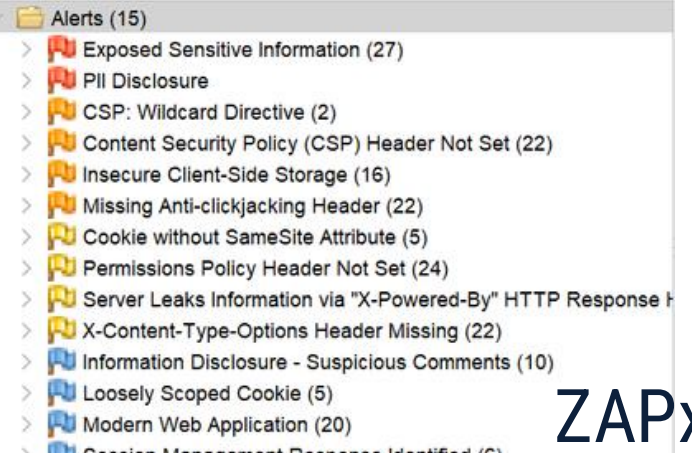
Vulnerable web application



Alerts (14)

- > PII Disclosure
- > CSP: Wildcard Directive (2)
- > Content Security Policy (CSP) Header Not Set (22)
- > Missing Anti-clickjacking Header (22)
- > Cookie without SameSite Attribute (11)
- > Permissions Policy Header Not Set (24)
- > Server Leaks Information via "X-Powered-By" HTTP Response
- > X-Content-Type-Options Header Missing (22)
- > Information Disclosure - Suspicious Comments (10)
- > Loosely Scoped Cookie (11)
- > Modern Web Application (20)
- > Session Management Response Identified (13)

ZAP

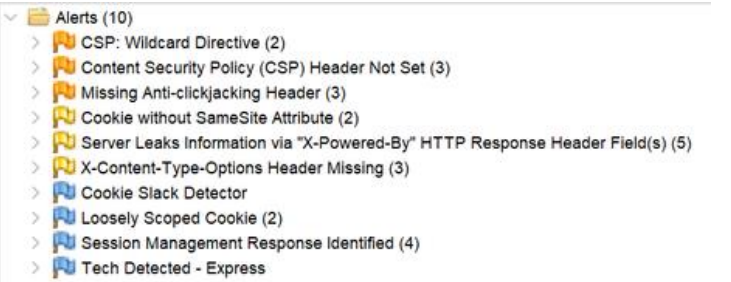


Alerts (15)

- > Exposed Sensitive Information (27)
- > PII Disclosure
- > CSP: Wildcard Directive (2)
- > Content Security Policy (CSP) Header Not Set (22)
- > Insecure Client-Side Storage (16)
- > Missing Anti-clickjacking Header (22)
- > Cookie without SameSite Attribute (5)
- > Permissions Policy Header Not Set (24)
- > Server Leaks Information via "X-Powered-By" HTTP Response
- > X-Content-Type-Options Header Missing (22)
- > Information Disclosure - Suspicious Comments (10)
- > Loosely Scoped Cookie (5)
- > Modern Web Application (20)
- > Session Management Response Identified (13)

ZAPx

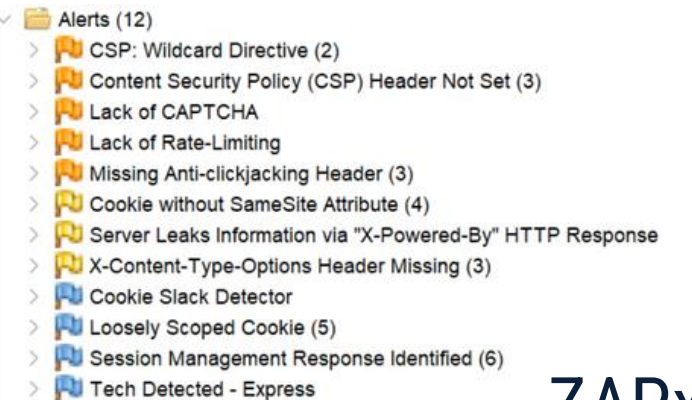
Client-side code vulnerability detection



Alerts (10)

- > CSP: Wildcard Directive (2)
- > Content Security Policy (CSP) Header Not Set (3)
- > Missing Anti-clickjacking Header (3)
- > Cookie without SameSite Attribute (2)
- > Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s) (5)
- > X-Content-Type-Options Header Missing (3)
- > Cookie Slack Detector
- > Loosely Scoped Cookie (2)
- > Session Management Response Identified (4)
- > Tech Detected - Express

ZAP



Alerts (12)

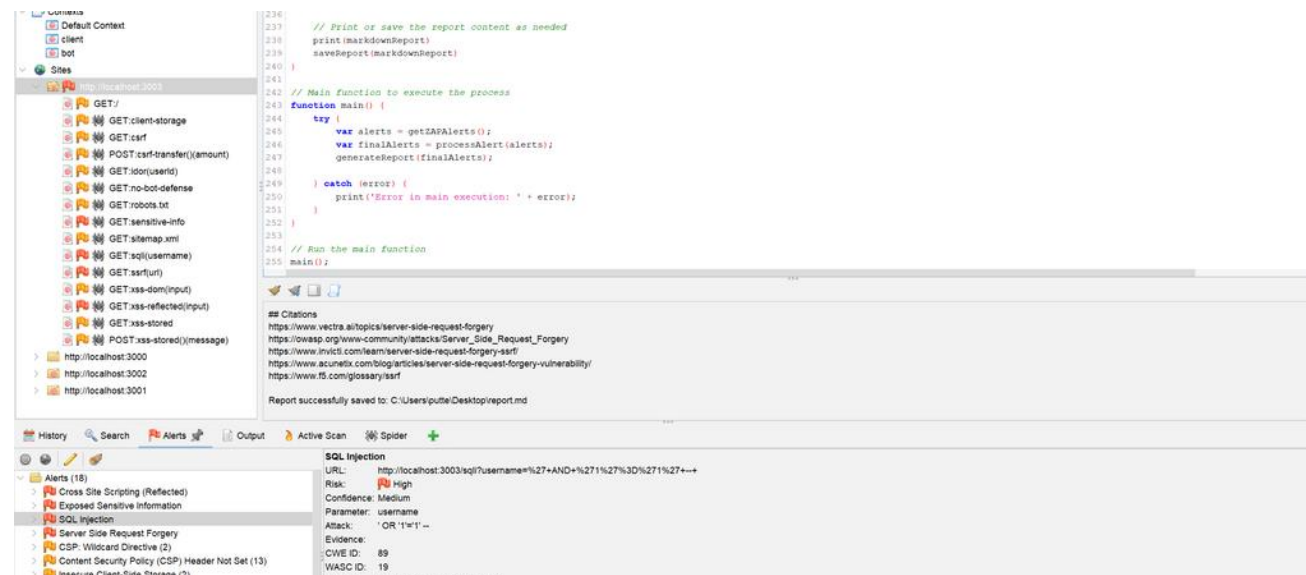
- > CSP: Wildcard Directive (2)
- > Content Security Policy (CSP) Header Not Set (3)
- > Lack of CAPTCHA
- > Lack of Rate-Limiting
- > Missing Anti-clickjacking Header (3)
- > Cookie without SameSite Attribute (4)
- > Server Leaks Information via "X-Powered-By" HTTP Response
- > X-Content-Type-Options Header Missing (3)
- > Cookie Slack Detector
- > Loosely Scoped Cookie (5)
- > Session Management Response Identified (6)
- > Tech Detected - Express

ZAPx

No Bot detection mechanism

TESTING OUR SCRIPTS (CONT.)

- **Generate remediation plans** using Stand Alone script



Mitigation Plan: Exposed Sensitive Information

ID: 3a81d99c-0f5d-4455-a16b-54e7de505754

Model: llama-3.1-sonar-small-128k-online

The provided mitigation plan is generally valid but could be refined to better align with the specific details of CWE-532 and the provided CWE information. Here's a revised version of the mitigation plan:

Vulnerability Explanation: CWE-532 - Insertion of Sensitive Information into Log File

What is CWE-532? CWE-532 is a vulnerability where sensitive information, such as API keys, passwords, or other confidential data, is inadvertently written to log files. This can expose critical information to unauthorized parties, potentially leading to security breaches or data misuse.

How It Works

1. **Logging Sensitive Data:** When an application logs sensitive information, it can include details like API keys, user credentials, or other confidential data.
2. **Unsecured Logs:** If these logs are not properly secured, they can be accessed by attackers, either directly or through interception.

Consequences

The consequences of an XSS attack can be severe:

- **Data Theft:** Attackers can steal sensitive user data such as session cookies, passwords, or credit card numbers.
- **Session Hijacking:** Attackers can hijack user sessions, allowing them to impersonate the user.
- **Malicious Actions:** Attackers can perform various malicious actions such as redirecting users to phishing sites or injecting malware into the user's browser.

Evidence from the Alert

The alert provides evidence of a reflected XSS attack:

```
{ "sourceid": "1", "other": "", "method": "GET", "evidence": "</p><script>alert(1);</script><p>", "pluginId": "40012", "cweid": "79", "confidence": "Medium", "wasid": "8", "description": "Cross-site Scripting (XSS) is an attack technique that involves echoing attacker-supplied code into a user's browser instance..." }
```

This indicates that the malicious code `</p><script>alert(1);</script><p>` was injected into the web application and reflected back to

Mitigation Strategies

To mitigate this vulnerability, several strategies can be employed:

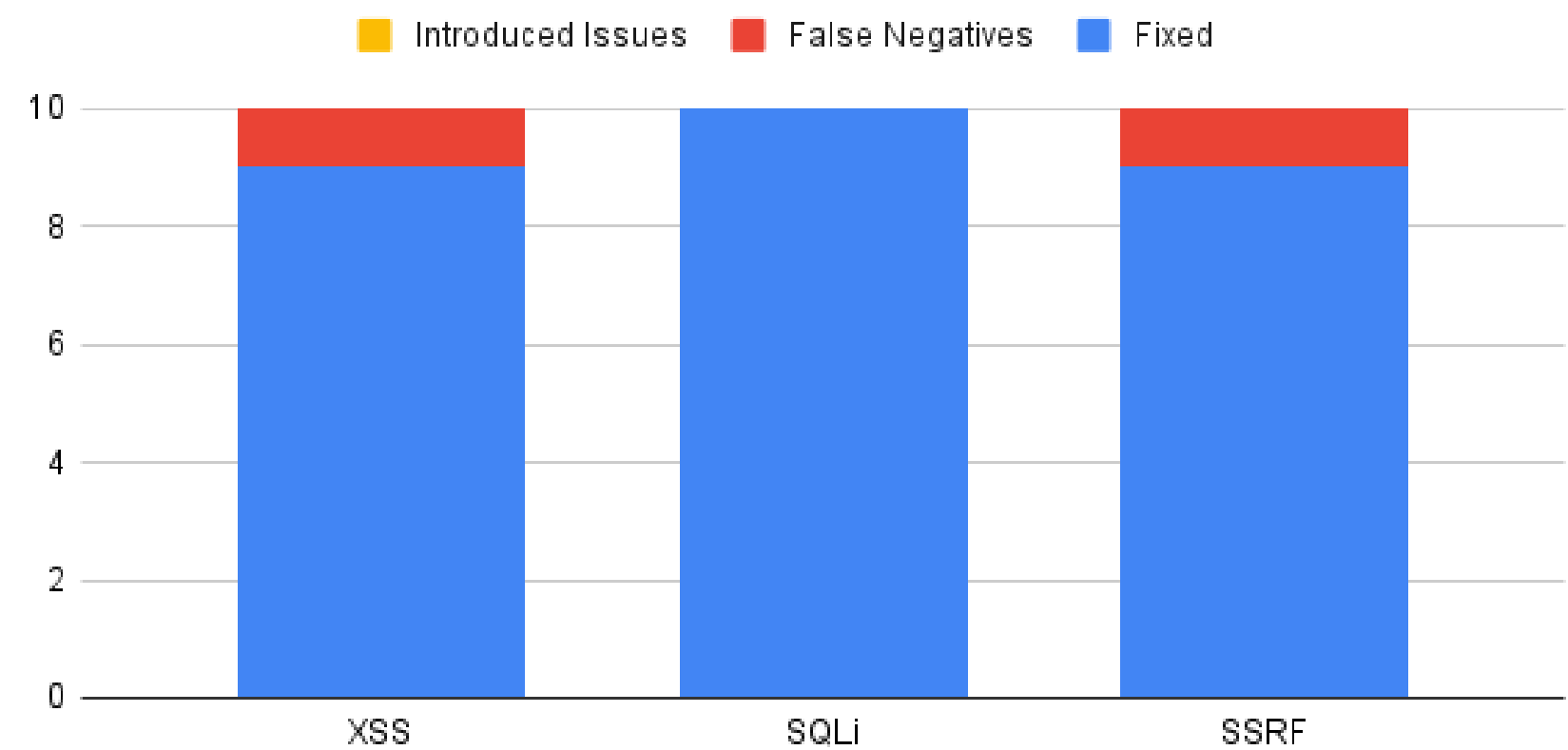
1. Use Parameterized Queries

◦ **Example (Node.js, Express, MySQL):**

```
const mysql = require('mysql');
const db = mysql.createConnection({
  host: 'localhost',
  user: 'user',
  password: 'password',
  database: 'database'
});

db.query('SELECT * FROM users WHERE username = ?', ['username'], (err, results) => {
  if (err) {
    console.error(err);
  } else {
    console.log(results);
  }
});
```

RESULTS



AI-powered remediation plan

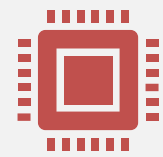
Metric for Passive Scan Rule	ZAP	ZAPx
Vulnerable Web Application		
Identified	✓	✓
Detected Instances	1	19
False Negative Instances	19	1
Patched Web Application		
False Positive Instances	0	2

Insecure client-side code

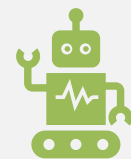
Metric for Active Scan Rule	ZAP	ZAPx
Vulnerable Web Application		
Identified	✗	✓
Detected Instances	0	20
False Negative Instances	20	0
Patched Web Application		
False Positive Instances	0	2

Bot defense mechanism

Discussion



The evaluation results show that ZAPx significantly enhances OWASP ZAP in both vulnerability detection and automated remediation, but several limitations remain.



1) AI-Generated Remediation Plans: Although achieving a high success rate (~93%), the AI occasionally produced generic or overlapping fixes for complex vulnerabilities such as advanced XSS and SSRF. Refining prompts, improving training data, and applying post-processing filters could enhance precision and clarity.



2) False Positives: ZAPx exhibited a slightly higher false-positive rate when scanning secured applications, mainly due to misclassified client-side issues or transient bot-defense responses. Incorporating adaptive learning and feedback mechanisms could improve reliability.



3) Limited Vulnerability Coverage: The current evaluation targets only five OWASP categories. Extending ZAPx with broader rule sets, external vulnerability databases, and complementary security tools will enable more comprehensive coverage.

CONCLUSION

- We propose extended ZAP functionalities:
 - AI-based mitigation plans
 - Lack of Bot defense mechanism detection
 - Insecure client-side code detection.
- Experimental results show that ZAPx achieves **higher detection accuracy and a 90% remediation success rate** compared to standard ZAP.

Future work will focus on refining AI guidance, incorporating adaptive learning, and broadening vulnerability coverage to establish ZAPx as a comprehensive tool for modern web application security.



References

1. <https://www.zaproxy.org>
2. <https://owasp.org>
3. K. Vimala and S. Fugkeaw, "VAPE-BRIDGE: Bridging OpenVAS Results for Automating Metasploit Framework," *2022 14th International Conference on Knowledge and Smart Technology (KST)*, Chon buri, Thailand, 2022, pp. 69-74, doi: 10.1109/KST53302.2022.9729085.
4. S. Fugkeaw, L. Hak, N. Ploysopond, W. Apichonkit and S. Lahankaew, "LightPEN: Optimizing the Vulnerability Exposures for Lightweight Penetration Test," *2023 15th International Conference on Knowledge and Smart Technology (KST)*, Phuket, Thailand, 2023, pp. 1-5, doi: 10.1109/KST57286.2023.10086896.
5. S. Alazmi and D. C. De Leon, "A Systematic Literature Review on the Characteristics and Effectiveness of Web Application Vulnerability Scanners," in *IEEE Access*, vol. 10, pp. 33200-33219, 2022, doi: 10.1109/ACCESS.2022.3161522.
6. A. Sudhodanan, R. Carbone, L. Compagna, N. Dolgin, A. Armando and U. Morelli, "Large-Scale Analysis & Detection of Authentication Cross-Site Request Forgeries," *2017 IEEE European Symposium on Security and Privacy (EuroS&P)*, Paris, France, 2017, pp. 350-365, doi: 10.1109/EuroSP.2017.45.
7. S. Alazmi and D. C. de Leon, "Customizing OWASP ZAP: A Proven Method for Detecting SQL Injection Vulnerabilities," *2023 IEEE 9th Intl Conference on Big Data Security on Cloud (BigDataSecurity), IEEE Intl Conference on High Performance and Smart Computing, (HPSC) and IEEE Intl Conference on Intelligent Data and Security (IDS)*, New York, NY, USA, 2023, pp. 102-106, doi: 10.1109/BigDataSecurity-HPIDS58521.2023.00028.
8. B. Zhang, X. Zhi, M. Wang, R. Ren and J. Dong, "Enhancing Java Web Application Security: Injection Vulnerability Detection via Interprocedural Analysis and Deep Learning," in *IEEE Transactions on Reliability*, vol. 74, no. 3, pp. 3642-3656, Sept. 2025, doi: 10.1109/TR.2024.3521381.
9. A. Dann, H. Plate, B. Hermann, S. E. Ponta and E. Bodden, "Identifying Challenges for OSS Vulnerability Scanners - A Study & Test Suite," in *IEEE Transactions on Software Engineering*, vol. 48, no. 9, pp. 3613-3625, 1 Sept. 2022, doi: 10.1109/TSE.2021.3101739.
10. ZAP Team, "OWASP ZAP," GitHub repository, [Online]. Available: <https://github.com/zaproxy/zaproxy>. [Accessed: Nov. 26, 2024].
11. I. Hwang, H. Cho and S. Kim, "Deriving Usability Evaluation Criteria for Threat Modeling Tools," in *IEEE Access*, vol. 13, pp. 65246-65265, 2025.
12. N. B. Chaleshtari, F. Pastore, A. Goknil and L. C. Briand, "Metamorphic Testing for Web System Security," in *IEEE Transactions on Software Engineering*, vol. 49, no. 6, pp. 3430-3471, 1 June 2023, doi: 10.1109/TSE.2023.3256322.
13. G. Floris *et al.*, "ModSec-AdvLearn: Countering Adversarial SQL Injections With Robust Machine Learning," in *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 6693-6705, 2025, doi: 10.1109/TIFS.2025.3583234.
14. N. M. Min, V. Visoottiviseth, S. Teerakanok and N. Yamai, "OWASP IoT Top 10 based Attack Dataset for Machine Learning," *2022 24th International Conference on Advanced Communication Technology (ICACT)*, PyeongChang Kwangwoon_Do, Korea, Republic of, 2022, pp. 317-322.
15. Suliman Alazmi and Daniel Conte De Leon. A systematic literature review on the characteristics and effectiveness of web application vulnerability scanners. *IEEE Access*, 2022.
16. P. Lachkov, L. Tawalbeh and S. Bhatt, "Vulnerability Assessment for Applications Security Through Penetration Simulation and Testing," in *Journal of Web Engineering*, vol. 21, no. 7, pp. 2187-2208, October 2022.
17. Node.js, "Node.js API Documentation," [Online]. Available: <https://nodejs.org/docs/latest/api/>. [Accessed: Nov. 26, 2024].
18. Express.js, "Express 5.x API Documentation," [Online]. Available: <https://expressjs.com/en/5x/api.html>. [Accessed: Nov. 26, 2024].



Thank you